

# **Modelos de Aprendizado de Máquina**

**Aula 03: Criação de Baselines**

# Agenda

Introdução a Criação de Baselines

**Abordagem Naive:** Como utilizar os dados de forma inocente

**Médias Móveis:**  
Realizando a criação de um baseline mais robusto

# Naive Simple: Características e Problema

- **Conceito Central:** Assume que o futuro imediato será exatamente igual ao passado imediato. A previsão de amanhã é o valor real de hoje.
- **Random Walk:** É o modelo estatisticamente ótimo para "Passeios Aleatórios", como cotações de ações na bolsa, onde a melhor estimativa de amanhã é o preço de fechamento de hoje.
- **Custo Computacional:** Nulo. Não requer treinamento, matrizes de correlação ou hiperparâmetros.

## 🎯 O Problema que Resolve

Ele estabelece o **Piso Mínimo de Acurácia (Benchmark Zero)**.

Se o seu modelo complexo de Deep Learning não conseguir bater as métricas de erro (MAE/RMSE) de um algoritmo que apenas repete o valor de ontem, seu modelo de ML é matematicamente inútil e sofre de *overfitting*.

# Naive Simple: Como Calcular

| Tempo (t)     | Vendas Reais (Y) | Previsão Naive ( $\hat{Y}$ ) | Erro Absoluto                     |
|---------------|------------------|------------------------------|-----------------------------------|
| Segunda       | 20               | -                            | -                                 |
| Terça         | 22               | 20                           | $ 22 - 20  = 2$                   |
| <b>Quarta</b> | <b>25</b>        | <b>22</b>                    | <b><math> 25 - 22  = 3</math></b> |
| Quinta        | 28               | 25                           | $ 28 - 25  = 3$                   |

Exemplo: A previsão para a Quarta-feira copia exatamente o valor real observado na Terça-feira.

## FÓRMULA: NAIVE SIMPLES

$$\hat{y}_{T+h|T} = y_T$$

Onde  $h$  é o horizonte (passos no futuro) e  $T$  é o instante atual.

# Naive Simple: Código Explicado

```
naive_simples.py

import pandas as pd

# Dataset fictício de vendas
df = pd.DataFrame({
    'vendas': [20, 22, 25, 28]
})

# Criando a previsão Naive Simple
# shift(1) translada a coluna 1 passo para baixo
df['prev_naive'] = df['vendas'].shift(1)

print(df)
# Retorno:
#   vendas  prev_naive
# 0     20         NaN
# 1     22        20.0
# 2     25        22.0
```

- **A função shift(1):** No Pandas, esta função empurra todos os dados de uma coluna uma linha para baixo (criando um atraso/lag).
- **Evitando Data Leakage:** O uso do shift garante que a previsão da linha atual só *tenha acesso* à informação da linha anterior, impedindo que o modelo enxergue o futuro.
- **O primeiro valor nulo (NaN):** Ocorre pois não há dados anteriores ao primeiro dia para realizar a previsão.

# Naive Sazonal: Características e Problema

- **Conceito Central:** Assume que o valor futuro será igual ao valor observado no mesmo ponto do ciclo sazonal anterior.
- **Aplicações Diretas:** Séries onde o calendário dita o comportamento.  
Ex: O tráfego de um restaurante no sábado será igual ao do último sábado; as vendas de panetone neste Natal serão iguais às do Natal passado.
- **Ignora a Tendência:** Não considera se a empresa está crescendo ou encolhendo ao longo do ano, apenas replica a sazonalidade em "estado bruto".

## O Problema que Resolve

Resolve a **ilusão preditiva em séries de alta sazonalidade**.

Se um Naive Simples for usado em vendas semanais, ele vai prever as altas vendas de Sábado usando a baixa de Sexta. O Naive Sazonal corrige isso imediatamente e serve como benchmark para modelos sazonais (SARIMA, Prophet).

# Naive Sazonal: Como Calcular

| Semana   | Dia da Semana | Real (Y) | Naive Sazonal (m=7) |
|----------|---------------|----------|---------------------|
| Semana 1 | Segunda       | 20       | -                   |
| Semana 1 | Terça         | 22       | -                   |
| Semana 1 | Domingo       | 45       | -                   |
| Semana 2 | Segunda       | 25       | 20 (Seg da Sem 1)   |
| Semana 2 | Terça         | 28       | 22 (Ter da Sem 1)   |

## FÓRMULA: NAIVE SAZONAL

$$\hat{y}_{T+h|T} = y_{T+h-m(k+1)}$$

Onde  $m$  é o tamanho do ciclo sazonal (ex: 7 para dados diários com ciclo semanal).

# Naive Sazonal: Código Explicado

```
naive_sazonal.py

import pandas as pd

# Dataset de 2 semanas de movimento
df = pd.DataFrame({
    'dia_semana': ['Seg', 'Ter', '...', 'Seg', 'Ter'],
    'vendas': [20, 22, 0, 25, 28] # Simplificado
})

# Definimos m=7 (ciclo semanal em dados diários)
m = 7

# Previsão copia o valor de 7 dias atrás
df['prev_saz_7'] = df['vendas'].shift(m)
```

- **Identificação do Parâmetro (m):** O cientista de dados deve conhecer o negócio para definir  $m$ . Dados mensais têm  $m=12$ , dados diários com ciclo semanal têm  $m=7$ .
- **A função `shift(m)`:** Ao invés de pular 1 linha, pulamos exatamente o bloco inteiro do ciclo (7 linhas).
- **Consequência Direta:** Os primeiros 7 dias da sua base de dados não terão previsão (ficarão como NaN), exigindo um histórico mínimo correspondente.

# Naive com Drift: Características e Problema

- **Conceito Central:** Equivalente a desenhar uma linha reta entre a primeira e a última observação histórica, e extrapolar essa taxa de crescimento para o futuro.
- **Tratamento de Tendência:** Ao contrário do Naive Simples que traça uma linha reta horizontal no horizonte de previsão, o Drift inclina a previsão com base na taxa média de mudança.
- **Risco:** É altamente sensível ao ponto inicial e final. Se o último dia registrado foi um *outlier*, a taxa de Drift será totalmente distorcida.

## O Problema que Resolve

Resolve a **limitação de séries com tendência direcional forte**.

Se um ativo sobe constantemente 1 unidade por dia, o Naive Simples vai sempre prever o valor defasado. O Drift corrige isso adicionando a "força de deslocamento" (Deriva) ao último valor conhecido.

# Naive com Drift: Como Calcular

- ◆ **Passo 1:** Calcule a variação total entre o primeiro e o último ponto.
- ◆ **Passo 2:** Divida pelo número de períodos (menos 1) para achar a inclinação.
- ◆ **Passo 3:** Some essa inclinação à previsão do Naive Simples.

*Exemplo:* Dia 1=10, Dia 5=30. Variação total=20. Passos=4. Drift =  $20/4 = +5$  **por dia**. A previsão para o Dia 6 será  $30 + 5 = 35$ .

**FÓRMULA: DRIFT (DERIVA)**

$$\hat{y}_{T+h|T} = y_T + h \left( \frac{y_T - y_1}{T-1} \right)$$

# Naive com Drift: Código Explicado

```
naive_drift.py

import pandas as pd

df = pd.DataFrame({
    'vendas': [10, 15, 20, 22, 30]
})

# 1. Isolando primeiro e último valor
y_t = df['vendas'].iloc[-1] # Último: 30
y_1 = df['vendas'].iloc[0]  # Primeiro: 10
T = len(df)                 # Qtd: 5

# 2. Calculando a inclinação (Drift)
drift_diario = (y_t - y_1) / (T - 1) # (30-10)/4 = 5

# 3. Aplicando sobre o Naive Simples
df['prev_drift'] = df['vendas'].shift(1) + drift_diario
```

- **iloc[-1] e iloc[0]:** Técnicas vitais do Pandas para capturar programaticamente as pontas da série temporal, independentemente do tamanho do *DataFrame*.
- **Vetorização:** Na linha final, ao somar a variável escalar `drift_diario` ao resultado da coluna gerada por `shift(1)`, o Pandas aplica a soma de forma otimizada linha a linha.
- **Limitação:** O Drift calculado assim assume uma taxa constante baseada em toda a história, o que pode falhar em séries que mudam de comportamento subitamente.

# Média Móvel Simples (SMA)

## = Características & Problema

Atribui o **mesmo peso** para todas as observações recentes. Resolve o problema do **ruído de alta frequência**, suavizando picos aleatórios diários para revelar a tendência central da série.

CÁLCULO: SMA (JANELA UNIDIRECIONAL)

$$\hat{y}_t = \frac{1}{m} \sum_{i=0}^{m-1} y_{t-i}$$

```
sma_baseline.py

import pandas as pd

# Serie de Vendas da Mecânica
df = pd.DataFrame({'vendas': [20, 22, 25, 28, 35, 50, 45]})

# 1. Calculando a SMA (Janela de 3)
df['sma_3'] = df['vendas'].rolling(window=3).mean()

# 2. Gerando o Forecast
# A previsão de amanhã é a SMA de hoje
df['prev_sma_3'] = df['sma_3'].shift(1)
```

rolling(3) agrupa as linhas ativas.

Sem o shift(1), haveria vazamento futuro.

# Média Móvel Ponderada (WMA)

## Características & Problema

Atribui **pesos decrescentes** lineares. O dado de ontem pesa mais que o de anteontem. Resolve o **problema do atraso (Lag)** da SMA, reagindo mais rápido a mudanças bruscas de tendência.

### CÁLCULO: WMA

$$\hat{y}_t = \frac{\sum_{i=1}^m i \cdot y_{t-m+i}}{\sum \text{pesos}}$$

```
wma_baseline.py

import numpy as np

# Definindo os pesos crescentes (ex: 1, 2, 3)
m = 3
pesos = np.arange(1, m + 1)

# Função Lambda para calcular o Produto Escalar
def wma(janela):
    return np.dot(janela, pesos) / pesos.sum()

# Aplicando WMA e transladando
s = df['vendas'].rolling(window=m)
df['prev_wma'] = s.apply(wma, raw=True).shift(1)
```

`np.dot()` multiplica arrays (valor x peso).

O `apply()` insere a função dentro da janela.

# Média Móvel Exponencial (EMA)

## ⚡ Características & Problema

Aplica um **decaimento exponencial** ( $\alpha$ ). Não descarta o histórico, mas seu impacto cai rapidamente rumo a zero. É a mais ágil das médias e **requer menos processamento** (complexidade  $O(1)$  recursiva).

CÁLCULO: EMA (RECURSIVA)

$$\hat{y}_t = \alpha y_t + (1 - \alpha) \hat{y}_{t-1}$$



ema\_baseline.py

```
# O Pandas possui suporte nativo à EMA via 'ewm'

# Calculando com alpha fixo (ex: 0.3)
alpha = 0.3
ema = df['vendas'].ewm(alpha=alpha, adjust=False)

# Também podemos calcular pelo span (janela)
# Alpha equivalente a span de 3: 2/(3+1) = 0.5
span = 3
ema_span = df['vendas'].ewm(span=span)

# Gerando Forecast (Sempre transladar!)
df['prev_ema'] = ema.mean().shift(1)
```

`adjust=False` garante a fórmula recursiva clássica.

Preferido em Forecasting por retenção de memória.

# Fluxo de Validação de Baseline

- **A Regra de Ouro:** Nunca avance para um modelo ARIMA, XGBoost ou LSTM sem antes tabular o MAE/RMSE das abordagens Naive e Médias Móveis.
- **Diagnóstico pelo Baseline:**
  - ◆ Se a série for ruidosa, a SMA será o melhor baseline.
  - ◆ Se tiver sazonalidade forte, o Naive Sazonal vai brilhar.
  - ◆ Se a dinâmica mudar rápido, a EMA sairá na frente.
- **Próximo Passo:** Como calcular as métricas de erro estatístico para comprovar matematicamente qual destas 6 abordagens é o vencedor provisório.



## Hands-on Pandas!

No laboratório, vocês aplicarão os códigos `shift()` e `rolling()` na base real tratada da MecâniQA, gerando as colunas preditivas necessárias para a próxima fase.

# **Modelos de Aprendizado de Máquina**

**Aula 03: Criação de Baselines**